CS/SE/CE 3354 Software Engineering

Dishcovery

A Matchmaking Software for Restaurants

Project Deliverable #2 (Part IV)

Project Deliverable #1 Section:

1. Dishcovery - URL : https://github.com/Larrison5/3354-Dishcovery.git

2. Delegations:

- Ryan Larrison - Created Github Repository(1.2), Invited members and TA(1.3), Assumptions(2.3), Feedback Response(2.4), Software Process Model(2.5),Requirements (2.6), Code
- Yisilamujiang Muhetaer - Commit and README file (1.4), Sequence diagram(8), Class diagram(9).
- Meghana Pula - Use Case Diagram(7)
- Shazil Bajwa - Project Scope Commit (1.5), Architectural Design(10)

3. Assumptions - Ryan Larrison

- Restaurant information is assumed to be accurate and regularly updated by restaurant owners or administrators.
- Restaurant recommendations are generated based on user preferences and available restaurant information. The system does not guarantee that every recommendation will perfectly match a user's preferences.
- User account information is assumed to comply with applicable U.S. privacy regulations

4. Feedback Response - Ryan Larrison

- There didn't seem to be any feedback provided. Unless the feedback would be located somewhere else but there was no feedback given for the Deliverable part 2, the proposal.

5.Software Process Model - Ryan Larrison

- We as a team decided that for our project, we will be using the **Incremental Process Model**. This model was chosen because Dishcovery can be developed in small increments rather than all at once. This allows our team to implement and test one feature before moving on to the next while also receiving feedback and making improvements during the development.
    - The planned development increments are:
        - Increment 1: User Registration and Login
        - Increment 2: Restaurant Search and Filtering
        - Increment 3: Restaurant Recommendation System
        - Increment 4: Favorites List and User Preference Management

- Increment 5: Testing, Bug Fixes, and User Interface Improvements

6. Requirements - Ryan Larrison

a. Functional Requirements

1. **User Registration and Login -** The system shall allows users to create an account, log in using their email and password, and securely access their personalized information.
2. **Restaurant Search -** The system shall allows users to search for restaurants by entering one or more search criteria, including restaurant name, cuisine, or location.
3. **Restaurant Filtering -** The system shall allows users to filter restaurant search results by cuisine type, price range, customer rating, distance, and dietary preferences.
4. **Restaurant Recommendation -** The system shall compares a user's dining preferences with restaurant information stored in the database and displays a ranked list of recommended restaurants.
5. **Restaurant Information -** The system shall displays detailed information for each restaurant, including its address, cuisine type, operating hours, menu, price range, customer rating, and contact information.
6. **Favorites Management -** The system shall allows authenticated users to save restaurants to a favorites list and remove restaurants from that list.
7. **Preference Management -** The system shall allows users to update their dining preferences so future restaurant recommendations better match their interests.

b. Non-Functional Requirements

Product Requirements

1. **Performance Requirements**
   - The system shall displays restaurant search results within 3 seconds.
2. **Space Requirements**
   - The system shall stores restaurant and user information in a database that supports at least 100 restaurants.
3. **Usability Requirements**
   - The system shall provides an easy-to-use interface that allows new users to perform a search without training.
4. **Efficiency Requirements**
   - The system shall efficiently processes user searches without unnecessary database operations.
5. **Dependability Requirements**
   - The system shall maintains at least 99% availability during normal operation.
6. **Security Requirements**
   - The system shall encrypts user passwords and requires authentication before accessing saved information.

Organizational Requirements

1. **Environmental Requirements**
   - The system shall operates on modern web browsers and mobile devices.

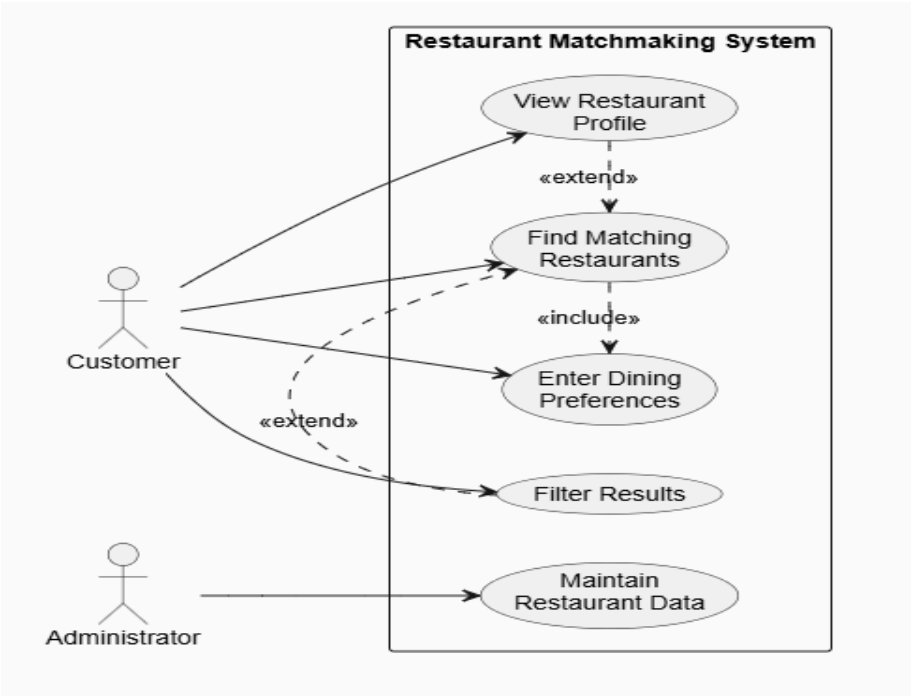
2. **Operational Requirements**
    - The system shall remains available except during scheduled maintenance.
3. **Development Requirements**
    - The software shall be developed using the Model-View-Controller (MVC) architecture and GitHub for version control.

External Requirements

1. **Regulatory Requirements**
    - The system shall comply with applicable privacy laws for collecting and storing user information.
2. **Ethical Requirements**
    - The system shall recommend restaurants based only on user preferences and restaurant data.
3. **Accounting Requirements**
    - If future payment features are added, the system shall maintain accurate transaction records.
4. **Safety/Security Requirements**
    - The system shall protect user accounts from unauthorized access.

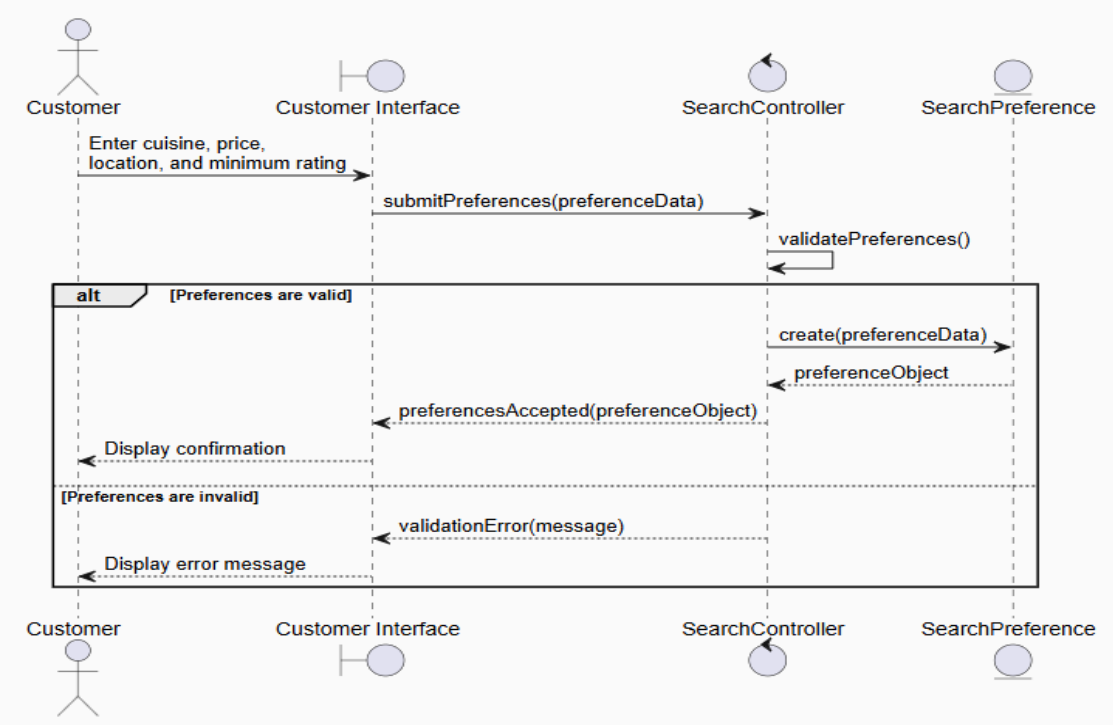
7. Use Case - Meghana Pula

Use Case 1	Dining Preferences	Customer
Use Case 2	Matching Restaurants	Customer
Use Case 3	Filter Results	Customer
Use Case 4	Restaurant Profiles	Customer
Use Case 5	Restaurant Data	Administrator

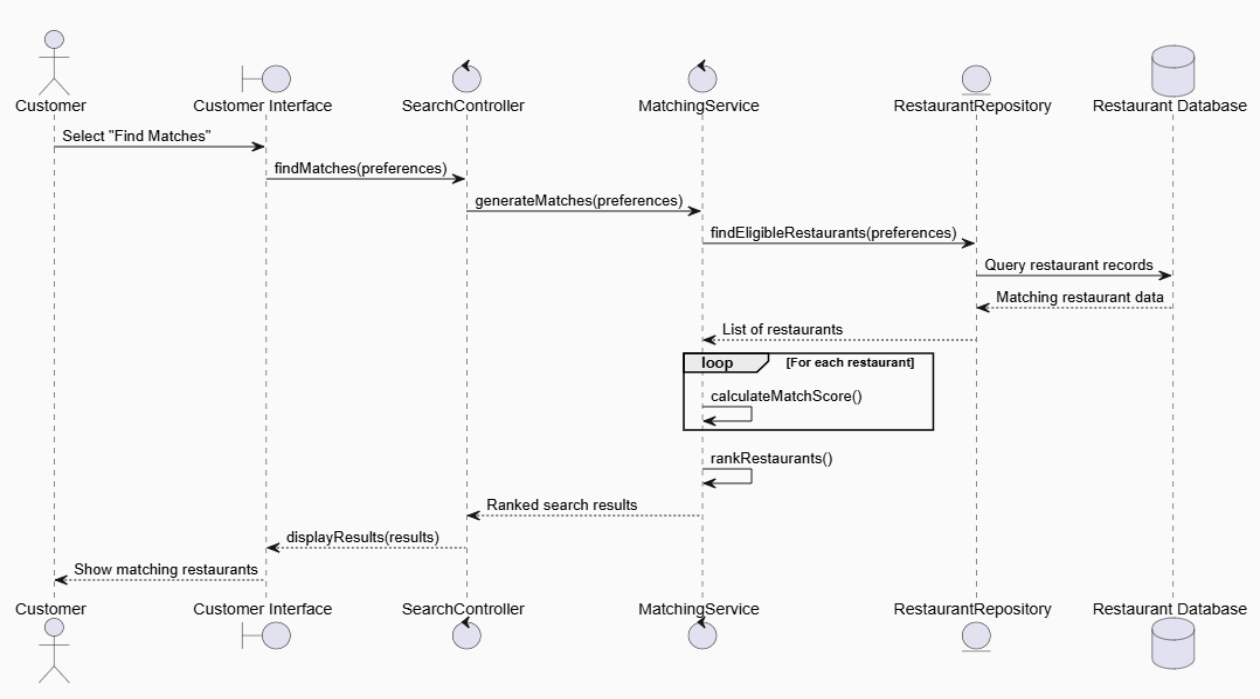


8. Sequence diagram - Yisilamujiang Muhetear

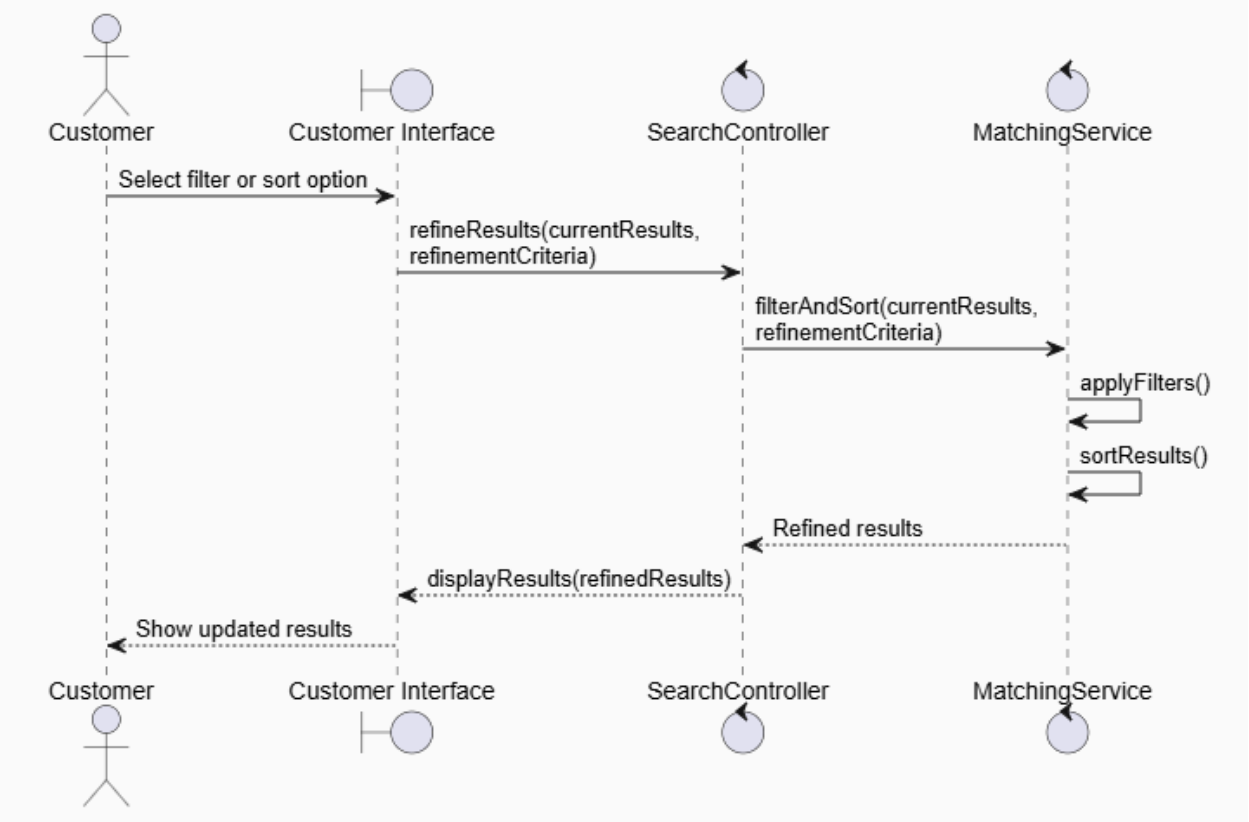
- Dining Preferences



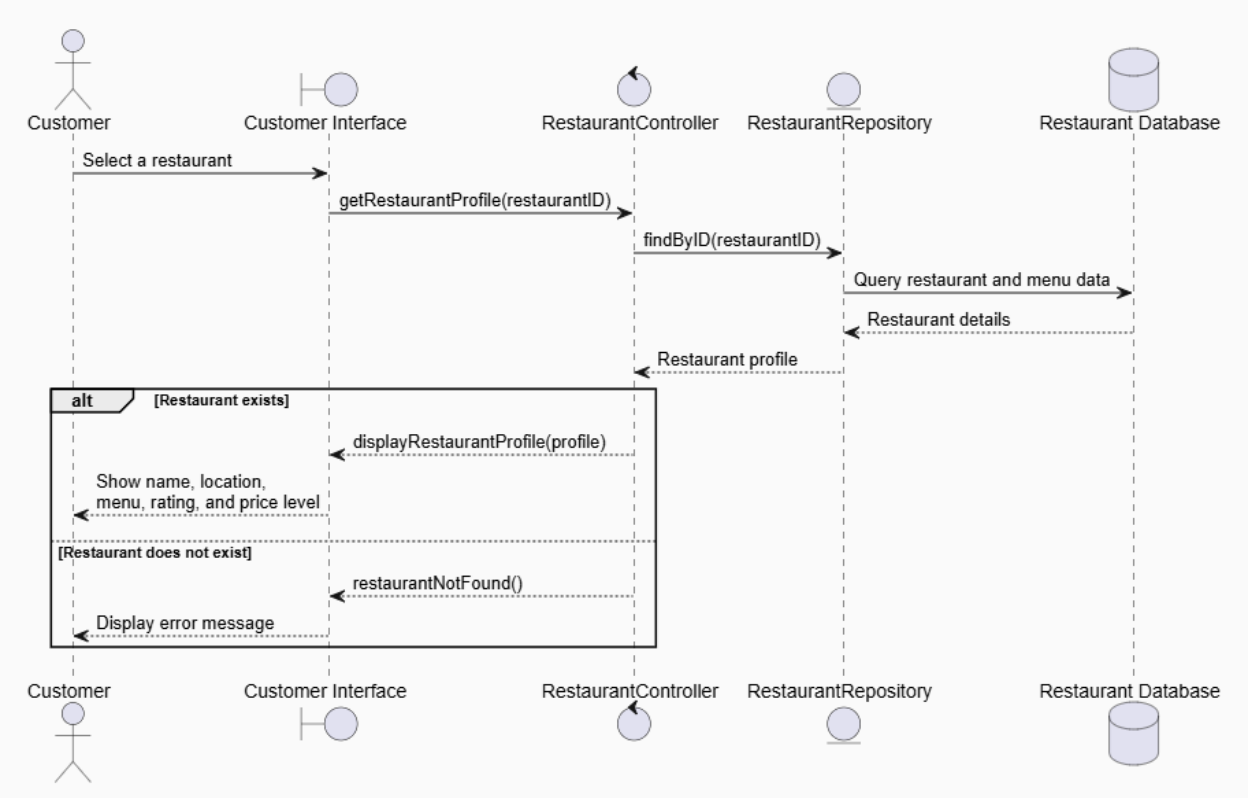
- Matching Restaurants



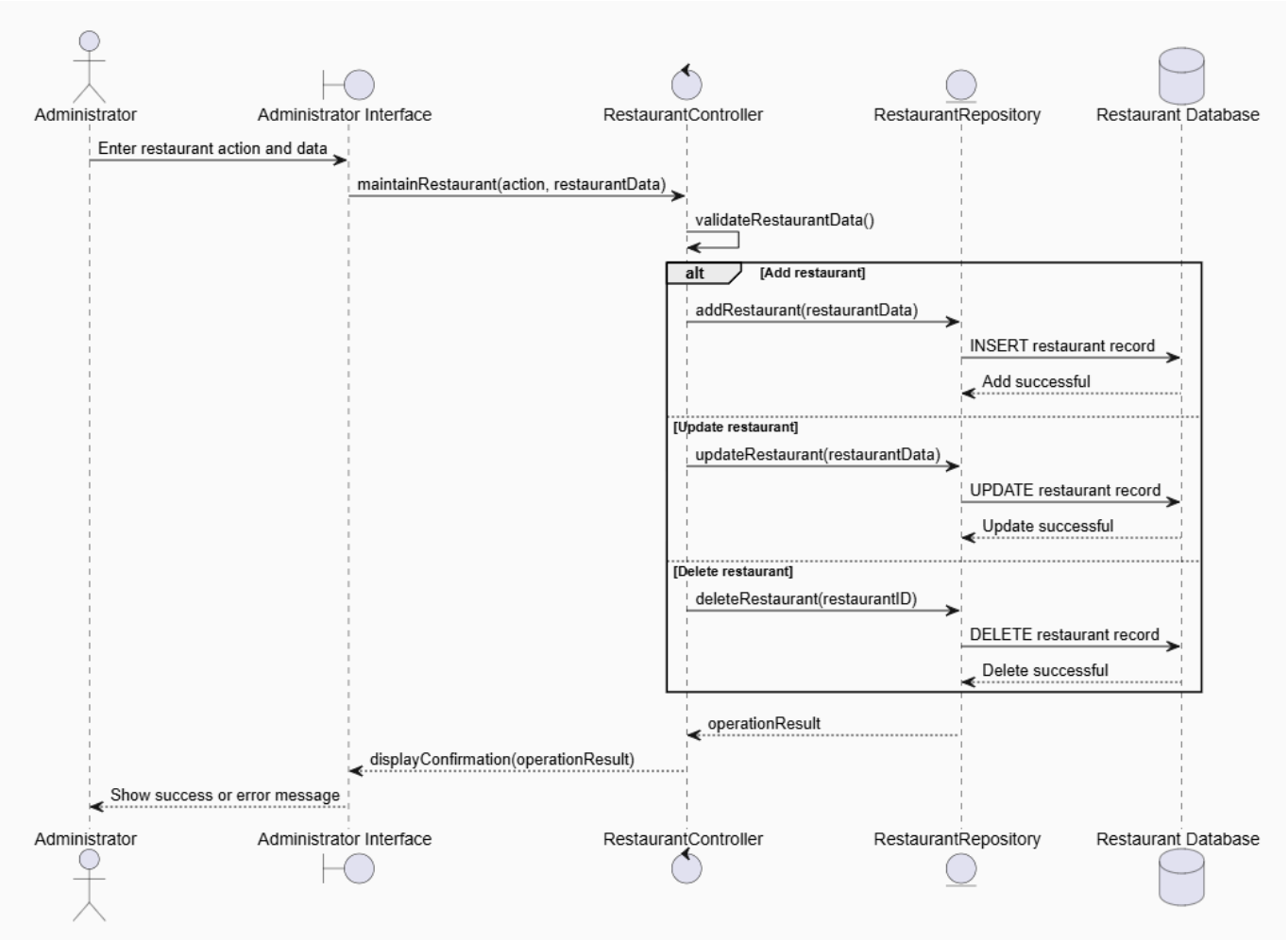
- Filter Results



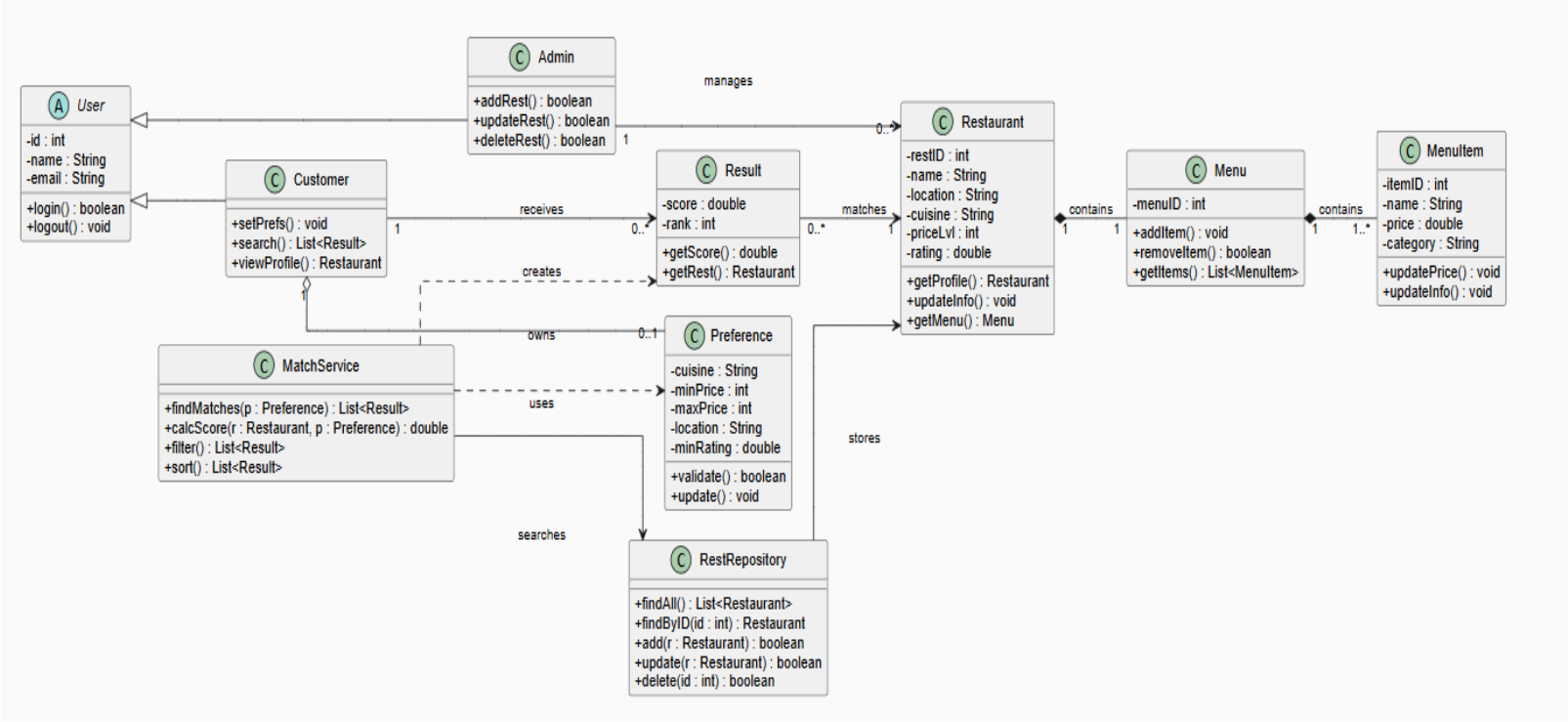
- Restaurant Profiles



- Restaurant Data


Administrator
Administrator Interface
RestaurantController
RestaurantRepository
Restaurant Database
Enter restaurant action and data
maintainRestaurant(action, restaurantData)
validateRestaurantData()
alt
[Add restaurant]
addRestaurant(restaurantData)
INSERT restaurant record
Add successful
[Update restaurant]
updateRestaurant(restaurantData)
UPDATE restaurant record
Update successful
[Delete restaurant]
deleteRestaurant(restaurantID)
DELETE restaurant record
Delete successful
operationResult
displayConfirmation(operationResult)
Show success or error message
Administrator
Administrator Interface
RestaurantController
RestaurantRepository
Restaurant Database

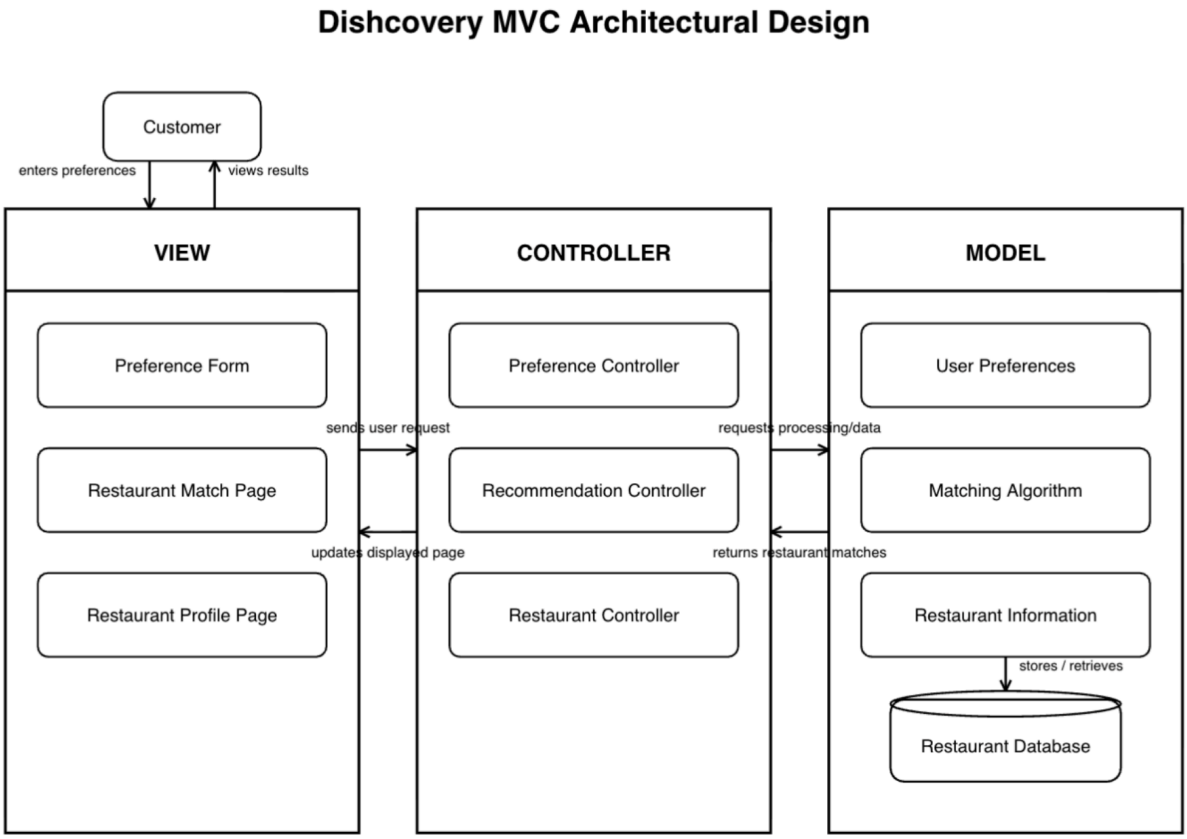
9. Class Diagram - Yisilamujiang Muhetear



10. Architectural Design- Shazil Bajwa

- For Dishcovery, we chose the Model-View-Controller (MVC) pattern. The Model will handle restaurant data, user preferences, and the matching system. The View will be the part the user sees, such as the preference page and restaurant results. The Controller will take the user's input and send it to the Model so the system can find matching restaurants.
- We chose MVC because it keeps the main parts of the program separate. This should make the project easier to organize and make changes to later.

Diagram:



END OF PROPOSAL DELIVERABLE #1:

PART 4:

a. Project Scheduling
   Start and End Date: 06/12-7/26
   i. The justification is the first group meeting happen on the 12 of June where we discussed what we should do for our project and started creating the Team Project Proposal (Part II) for Dishcovery. The end date coincides with the due date of this report.
   ii. Are Weekends Counted?
      1. Yes, weekends are included in the project schedule. Since this was a semester long team project, work was completed whenever team members were available, which include the weekends.
      2. The average amount of time spent was around 3 hours per day. However, the amount of time varied depending on the project phase. During planning and documentation, few

hours was required, while implementation, testing, and preparing took most of the time. Therefore, the average over the whole project was around 3 hours per day.

b. Cost, Effort and Pricing Estimation:
   i. The Function Point Method was selected to estimate the effort and cost of making Dishcovery. Function Point analysis estimates teh functional size of a software system by counting the inputs, outputs, queries, data files, and external interfaces used by the application. This aligns with Discovery which has clear defined user functions, database tables, and system interactions.

Function category	Count	Complexity	Weight	Count × Weight
User inputs	10	Average	4	40
User outputs	9	Average	5	45
User queries	6	Average	4	24
Data files/tables	4	Average	10	40
External interfaces	0	Average	7	

   ii. GFP = 40+45+24+40+0 = 149

c. Estimate Cost of Hardware Products:

Hardware product	Quantity	Estimated cost per item	Total
Development laptops	4	$900	$3,600
Mobile device for responsive testing	1	$300	$300
External backup drive	1	$100	$100
Networking and miscellaneous equipment	1	$100	$100
Estimated hardware total			**$4,100**

i. The estimated hardware cost for Dishcovery is $4,100. Most of the cost comes from the 4 laptops that is required by each member of the team.

d. Dishcovery was developed mainly with free and open-source software. Python, Flask, SQLite, SQLAlchemy, Visual Studio Code, Git, and GitHub do not require paid licenses for this project. However, a production version would require web hosting and a domain name.
   i. Domain name for One year = $20, Cloud Hosting = $300, Total = $320

e. The personnel estimate is based on the staff that would be needed to professionally develop Dishcovery.
The proposed development staff includes:

- One project manager and business analyst
- Two software developers
- One user-interface designer
- One software tester
- One employee responsible for customer training after installation

Personnel role	**Estimated hours**	**Hourly rate**	**Estimated cost**
Project manager/business analyst	90	$45	$4,050
Software developer 1	160	$40	$6,400
Software developer 2	160	$40	$6,400
User-interface designer	60	$35	$2,100
Software tester	80	$32	$2,560
Installation and user training	8	$35	$280
Estimated personnel total	**558**		**$21,790**

i. The estimated personnel cost for Dishcovery is $21,790. The two software developers account for most of the personnel cost because they are responsible for implementing the application, database, user authentication, recommendation functions, administrator tools, and other major features. The estimate also

includes time for project planning, interface design, testing, installation, and training. Training is expected to be limited because Dishcovery uses a straightforward web-based interface.

Part 5: Test Plan

a. The purpose of testing Dishcovery was to verify that the application's core features function correctly and meet the project requirements. Testing focused on user authentication, restaurant searching, personalized recommendations, favorites management, and administrator functions. Unit testing was used to validate individual components, while manual testing was performed to ensure the system behaved correctly from the user's perspective. The following objectives were established for testing Dishcovery:
   - Verify that users can successfully register and log into the system.
   - Verify that restaurant searches return accurate results.
   - Verify that personalized recommendations are generated based on user preferences.
   - Verify that users can add and remove restaurants from their favorites.
   - Verify that administrator functions correctly add, update, and delete restaurant records.
   - Verify that invalid input is handled appropriately through validation and error messages.

b. The MatchingService.score() method was selected as the unit under test. Automated testing was performed using the Python pytest framework. Two unit tests were created to verify the recommendation algorithm. The first test verifies that a restaurant meeting all customer preferences receives the expected recommendation score. The second test verifies that a restaurant with little or no match receives a significantly lower score. The automated test results confirmed that the recommendation algorithm functions correctly.

c.

```
collected 2 items

tests/test_matching_service.py::test_restaurant_matches_all_preferences PASSED                    [ 50%]
tests/test_matching_service.py::test_restaurant_does_not_match_preferences FAILED                 [100%]

============================================ FAILURES ============================================
__________________________ test_restaurant_does_not_match_preferences ___________________________

    def test_restaurant_does_not_match_preferences():
            restaurant = FakeRestaurant(
                cuisine="Italian",
                location="Austin",
                price_level=4,
                rating=3.0,
                dietary_options="None"
            )

            preferences = FakePreferences(
                cuisine="Korean",
                location="Dallas",
                max_price_level=2,
                minimum_rating=4.0,
                dietary_options="Vegetarian"
            )

            result = MatchingService.score(restaurant, preferences)

>           assert result.score == 100
E           assert 3.0 == 100
E            +  where 3.0 = MatchResult(restaurant=<test_matching_service.FakeRestaurant object at 0x000002646D915590>, score=3.0, reasons=[]).score

tests\test_matching_service.py:76: AssertionError
==================================== short test summary info =====================================
FAILED tests/test_matching_service.py::test_restaurant_does_not_match_preferences - assert 3.0 == 100
=================================== 1 failed, 1 passed in 0.41s ==================================
```

Part 6:

Dishcovery shares similarities with several existing restaurant recommendation platforms, including Yelp, Google Maps, and TripAdvisor. These applications allow users to search for restaurants, read reviews, filter results, and view restaurant information. Like these systems, Dishcovery helps users discover restaurants based on their preferences and provides detailed restaurant information to support decision-making.

One difference is that Dishcovery places a greater emphasis on personalized recommendations through its matching algorithm. Instead of primarily relying on user reviews or popularity rankings, Dishcovery calculates a recommendation score by comparing a customer's preferred cuisine, location, price range, minimum rating, and dietary preferences with restaurant attributes. This allows recommendations to be tailored more closely to each user's individual preferences.

Compared to Yelp and Google Maps, Dishcovery is a simplified academic software engineering project. It does not currently integrate real-time restaurant information, customer reviews, GPS navigation, or external APIs. However, its modular architecture and recommendation algorithm provide a solid foundation for adding these features in future versions. By focusing on preference-based matching, Dishcovery demonstrates how software engineering principles can be applied to build an effective recommendation system while remaining easier to design, implement, and test than large commercial applications.

Cite: Yelp. *Yelp*. https://www.yelp.com/

Google. *Google Maps*. https://maps.google.com/

TripAdvisor. *TripAdvisor*. https://www.tripadvisor.com/

Conclusion:

The Dishcovery project successfully demonstrates the design and implementation of a restaurant recommendation system that helps users find restaurants based on their dining preferences. Throughout this project, our team followed the Incremental Process Model to develop the system in manageable stages, allowing features to be implemented, tested, and improved over time. We completed the system design, estimated the project's cost and effort, developed a testing plan, and verified the recommendation algorithm using automated unit testing with the pytest framework.

Overall, the project met its functional requirements by providing user authentication, restaurant recommendations, favorites management, and administrator functions. The automated tests confirmed that the recommendation algorithm correctly evaluates both matching and non-matching restaurant preferences. While additional features such as real-time restaurant data and external API integration could further improve the application, the current implementation provides a strong foundation for future development. Dishcovery demonstrates how software engineering principles, planning, testing, and documentation can be applied to develop a practical and user-friendly recommendation system.

References:

[1] Google, "Places API Overview," Google for Developers. [Online]. Available: https://developers.google.com/maps/documentation/places/web-service/overview. [Accessed: Jul. 27, 2026].

[2] Google, "REST Resource: places," Google for Developers. [Online]. Available: https://developers.google.com/maps/documentation/places/web-service/reference/rest/v1/places. [Accessed: Jul. 27, 2026].

[3] Yelp Inc., "Getting Started with the Yelp Places API," Yelp for Developers. [Online]. Available: https://docs.developer.yelp.com/docs/places-intro. [Accessed: Jul. 27, 2026].

[4] Yelp Inc., "Yelp Places API Overview," Yelp for Developers. [Online]. Available: https://docs.developer.yelp.com/docs/overview. [Accessed: Jul. 27, 2026].